PRODUCT BLUEPRINT / PRIVATE ALPHA → PUBLIC PLATFORM

STXIC Realm

A space that’s yours. | A customizable communication platform with personal pixel homes and optional community halls.

Version 1.0 • 26 October 2026 • Status: planning specification

Initial audience: five friends | Initial operating budget target: ₱0/month | Platforms: Windows and Web

Document status: The product direction is agreed in principle. Technical choices, schedules, capacities and interface details are proposals pending implementation and testing.

Contents

01 Executive summary and product strategy

02 Audience, use cases and requirements

03 Brand identity and visual design system

04 Experience architecture and navigation

05 Personal pixel homes

06 Optional server halls

07 Pixel-world interactions and multiplayer

08 Identity Studio and theme engine

09 Messaging, friends and presence

10 System architecture and repository

11 Database model and access control

12 Security, privacy and moderation

13 Free-tier deployment and operational limits

14 API, realtime events and data contracts

15 Accessibility, performance and quality

16 Delivery roadmap and acceptance criteria

17 Public launch and scaling strategy

18 Risks, decisions and open questions

19 Appendices: schemas, sample payloads and glossary

1. Executive summary and product strategy

STXIC Realm is a desktop-and-web social communication platform combining familiar direct messaging, group channels and voice-ready architecture with a distinctive interactive identity layer: each member has a visitable 2D pixel-art home; a server owner may optionally enable a shared Server Hall.

The product is not a game replacing chat. Text communication remains fast and accessible through conventional UI, while pixel spaces offer a more personal, spatial way to socialize and discover content.

Positioning: “Your group’s digital home.” Brand tagline: “A space that’s yours.” Primary differentiator: meaningful interactive homes and halls connected to social identity, not merely cosmetic themes.

North-star test: do the original five members voluntarily return to their homes/hall, invite each other, and use interactive objects instead of treating the pixel world as a one-time novelty?

Product principles

- Communication first: core chat never depends on loading the game scene.
- Identity belongs to the user: expressive profile, avatar and home customization.
- Optional spaces: server halls are opt-in and may be disabled.
- Low-cost by design: static assets, bounded storage, modest realtime update rates.
- Privacy and accessibility are default requirements, not later add-ons.
- Scale by measured demand, not premature infrastructure complexity.

Success measures

Metric	Alpha target / method
Weekly active testers	All five invited testers; monitor actual use
Home engagement	At least three testers visit or edit homes weekly (hypothesis)
Chat reliability	No lost acknowledged messages in testing
Visit reliability	Permission checks enforced; join/leave sync works
Cost	Remain within free plan allowances
Quality	Keyboard-friendly non-pixel path for essential features

2. Audience, use cases and requirements

Primary users: five invited friends, typically communicating, gaming, coding or hanging out. Future users: public friend groups and community operators. Roles: guest (unauthenticated), member, home owner, space member, moderator, administrator and platform operator.

Core journeys: create account via invite; set up identity; decorate home; invite/visit friend; open chat or guestbook from home; join server; enter optional hall; chat from normal interface; manage privacy and permissions.

Scope tiers

Priority	In scope
P0 / Alpha	Invite-only auth, profiles, friends, DMs, spaces/channels, persistent text chat, home editor, visits, avatars, basic furniture, optional hall, simple interactive objects
P1 / Beta	Richer themes, room templates, memory wall, event board, presence refinements, stronger moderation, asset uploads, voice pilot
P2 / Future	Neighborhoods, public discovery, creator assets, plugin/blocks platform, pets, minigames, multiroom homes

Non-goals for alpha

- Full 3D simulation or open-world MMO.
- Unlimited user-uploaded scripts, HTML or CSS.
- Payments, subscriptions, cosmetic marketplace or token economy.
- Production-grade video calls, livestreaming or game overlays.
- Public sign-up without moderation capacity.

3. Brand identity and visual design system

Brand architecture: STXIC is the signature umbrella; Realm is the product. Personality: welcoming, calm, expressive, modern and technically competent. Voice: clear, friendly, concise; avoid gamer-only jargon.

Logo direction: a simple orbit/portal motif suggesting identity and connection. A bespoke mark must be designed and trademark availability checked before public release; conceptual iconography is not final artwork.

Token	Default
Background	Obsidian #101115
Surface	Graphite #1A1C22
Elevated	Slate #262933
Accent	Realm Violet #A78BFA
Primary text	Mist #ECECF1
Secondary text	Muted #9295A3
Typography	Inter; Segoe UI Variable fallback; JetBrains Mono for code
Spacing	4, 8, 12, 16, 24, 32 px
Radius	8, 12, 16, 24 px
Motion	120–240 ms UI transitions; reduced-motion override

Visual composition

- Quiet, restrained default shell; users customize their own accents and themes.
- Pixel art uses a consistent grid, nearest-neighbor scaling, restrained palette and legible silhouettes.
- Do not let avatar effects obscure names, controls, moderation labels or message text.
- Offer light and dark themes and sufficient contrast for body text and interactive states.

4. Experience architecture and navigation

Primary desktop shell: space rail, channel/friend sidebar, main chat or pixel-world canvas, optional member/activity inspector, bottom user control strip. Pixel scenes open as dedicated views with fast switching back to chat.

Main routes: /friends, /messages/:conversationId, /spaces/:spaceId/channels/:channelId, /homes/:userId, /spaces/:spaceId/hall, /studio/profile, /studio/home, /settings. Desktop and web share the same application-level routes.

Mobile-responsive browser support is desirable, but a polished phone application is outside alpha scope. A touch-friendly room inspection mode may precede full touch editing.

Essential states

- Loading, empty, offline, permission denied, private home, full room, reconnecting, upload rejected and conflict resolution.
- Every spatial interaction has a text/list-based equivalent for accessibility and low-end devices.
- Unsaved edits are visibly marked; saving and publishing are separate actions if preview mode is supported.

5. Personal pixel homes

Each profile owns one personal home in alpha. A home is a bounded 2D tile-grid scene with fixed walls/floors, a configurable furniture layer and defined entry spawn. The home is stored as data, not a generated screenshot.

Visitors enter via a profile Visit Home action. Home privacy: private, friends-only, invite-only, public (public option disabled for alpha unless explicitly enabled). Owner may set visitor capacity, guestbook permissions and whether presence is shown.

Editor workflow: open editor → choose tile/furniture → place/move/rotate where supported → validate collision and budget → preview → save. Server validates ownership, allowed asset IDs, positions and object counts.

Alpha furniture and interactions

Object	Behavior
Desk / computer	Opens owner-configured links or project card
Guestbook	Read/write moderated messages per privacy setting
Photo frame	Displays approved uploaded or bundled image
Sofa / chair	Avatar sit animation; optional chat shortcut
Music player	Opens a playlist link; synchronized playback

	deferred
Portal / door	Return to chat or navigate to allowed hall
Trophy shelf	Shows manually selected profile achievements
Plant / lamp / rug	Decorative only; low-cost static render

World rules

- Start with a fixed small map such as 16×12 tiles; confirm through playtesting.
- Grid positions are integer coordinates; object anchors, rotation and collision footprints are validated.
- A user can move only objects they own or have explicit edit rights for.
- Use server-side limits on objects, image size, guestbook posts and concurrent visitors.
- Avoid copyright-infringing game sprites; create or license assets for commercial redistribution.

6. Optional server halls

Every server can exist without a hall. An owner may enable one hall from a template; enabling creates a hall scene associated with the server. Disabling hides access while retaining the layout until explicitly deleted.

Hall roles: owner manages feature and permissions; authorized decorators edit layout; moderators manage content and visitors; ordinary members may enter and interact; guests only if permitted.

Possible templates: Gaming Lounge, Guild Hall, Café, Tech Lab. Initial implementation should ship one neutral starter template plus a small curated furniture catalog.

Interactive hall objects: channel portal, announcement board, event board, community memory wall, member showcase and welcome sign. Channel access checks remain mandatory when a portal is clicked.

Hall permissions

Action	Owner	Decorator	Member	Visitor
Enter hall	Yes	Yes	Yes	If permitted
Edit layout	Yes	Yes	No	No
Configure channel portal	Yes	Role-dependent	No	No
Moderate guestbook	Yes	If granted	No	No
Enable/disable hall	Yes	No	No	No

7. Pixel-world interactions and multiplayer

Render using Phaser 3 embedded in React. Use Tiled for map authoring and LibreSprite or other properly licensed pixel tools for sprite production. Browser and Tauri share renderer code.

For five users, synchronize coarse movement and presence via Supabase Realtime. The client renders movement smoothly between updates; do not broadcast every animation frame. Realtime presence is ephemeral, not the authoritative source for account membership or permissions.

Room join protocol: authenticate → authorize room → load scene version → subscribe to presence/events → announce join → reconcile current occupants → process bounded position updates → leave/timeout. Validate claimed positions, speed and bounds; clients must not trust other clients blindly.

Public scale may require an authoritative dedicated room server (e.g., Colyseus or custom WebSocket service), especially for high-frequency movement, anti-cheat or large rooms. Supabase is a prototype path, not a guarantee of MMO-scale simulation.

Event taxonomy

Event	Payload outline
room.join	roomId, sessionId, spawn
player.move	roomId, userId, x, y, direction, sequence
player.leave	roomId, userId
object.interact	roomId, objectId, action
room.layout.updated	roomId, version
presence.changed	userId, state, lastSeen

8. Identity Studio and theme engine

The visual identity system has two separate layers: public profile appearance (avatar frame, banner, badges, room) and private local interface theme (colors, fonts, density, panels). A user's theme does not silently restyle another user's application.

Theme pipeline: editor controls → schema-validated versioned JSON → safe token mapping → CSS variables and approved effect components → live preview. No imported executable JavaScript. Custom CSS, if ever added, requires isolation and an explicit threat model.

Profile asset upload rules: MIME/type verification, dimension and size limits, safe rendering, metadata stripping where possible, storage quotas and moderation hooks. Prefer animated WebP or spritesheets over large GIFs.

Theme token families

- Colors: background, surface, elevated, text, muted, accent, border, success, warning, danger.
- Typography: allowlisted family, scale and line height.
- Layout: compact/comfortable density, sidebar width bounds, bubble mode, panel visibility.
- Effects: avatar frame preset, glow intensity, animation toggle, reduced-motion behavior.
- Portability: schemaVersion, preset ID, creator ID, import/export, safe fallback.

9. Messaging, friends and presence

Persistent text chat must function independently of pixel scenes. Support friend requests, DMs, space text channels, message history, edits/deletes, reactions, basic attachments and unread counts. Server-side authorization checks every write and read.

Messages are committed to PostgreSQL before clients display delivery success. Realtime subscriptions inform clients of new data; on reconnect, clients fetch missed records using cursors. Use stable message IDs and idempotency keys to prevent accidental duplicates.

Presence states: online, away, do not disturb, invisible. Pixel-home presence and activity sharing are opt-in and must not expose private application or game details by default.

Voice/video: defer from alpha. Evaluate LiveKit or an alternative once the five-user text-and-pixel experience is proven; free participant-minute limits may be insufficient for regular group calls.

10. System architecture and repository

Use a pnpm workspace monorepo with apps/web (React + Vite) and apps/desktop (Tauri shell), packages/ui, packages/themes, packages/world, packages/features, packages/data and packages/shared. Supabase migrations and policies are version-controlled.

Backend MVP: Supabase Auth, PostgreSQL, Realtime and Storage; Cloudflare Pages for web delivery; GitHub Actions for CI; GitHub Releases for early Windows builds. Avoid running a dedicated paid server initially.

Keep service calls behind repositories/adapters: AuthService, ChatRepository, WorldRepository, StorageService and RealtimeTransport. This lowers migration cost if public traffic later requires different infrastructure.

Repository outline

apps/web/ | apps/desktop/src-tauri/ | packages/ui/ | packages/themes/ | packages/world/ | packages/features/ | packages/data/ | supabase/migrations/ | docs/ | .github/workflows/

Architecture boundaries

- React owns navigation, settings, forms and accessible messaging UI.
- Phaser owns tile maps, sprites, collision and spatial rendering.
- Domain services own permissions, state changes and serialization.
- Supabase is not called directly from arbitrary presentation components.
- Public secrets and service-role credentials must never be bundled in client applications.

11. Database model and access control

All primary keys are UUIDs; include created_at and updated_at where applicable. Use foreign keys, indexes, constraints and row-level security. Keep private account data separate from public profile metadata.

Table	Key fields / purpose
profiles	user_id, handle, display_name, bio, avatar_asset_id
friendships	requester_id, addressee_id, status
spaces	id, owner_id, name, visibility
space_members	space_id, user_id, role
channels	id, space_id, type, name
conversations / participants	DM group and member access

messages	id, channel_or_conversation, author_id, body, edited_at
reactions	message_id, user_id, emoji
homes	owner_id, scene_id, privacy, capacity
halls	space_id, scene_id, enabled
scenes	id, kind, schema_version, layout_json, revision
scene_objects	scene_id, asset_id, x, y, rotation, config_json
assets	owner_id, storage_key, mime, bytes, status
profile_appearances	user_id, frame, banner, effects_json
user_themes	user_id, schema_version, theme_json
guestbook_entries	home_id, author_id, body, status
invites	scope, code_hash, expiry, max_uses

Authorization invariants

- A profile owner may edit only their own profile, theme and home.
- A channel message is readable only by authorized participants or members.
- A hall decorator must have an active role grant; frontend visibility is not authorization.
- Invite codes are stored hashed where feasible and are limited by expiry and use count.
- Storage paths and signed URLs are restricted according to asset visibility.
- A blocked user cannot bypass restrictions via home visits, guestbooks or invites.

12. Security, privacy and moderation

Adopt least privilege and deny-by-default row-level security. Test policies with multiple user identities, not only a service account. Add rate limits for sign-in, invitations, chat, guestbooks, uploads and movement updates. Protect sensitive mutations with server-side validation or database functions.

Personal homes can reveal social activity. Default new homes to friends-only for alpha; require explicit consent before public discovery. Include block/report controls, privacy settings and deletion/export paths.

For a public release, implement reporting workflows, moderator queues, content takedown, retention policy, abuse response, age-related policy decisions, terms and privacy notice before opening signups.

Threat model snapshot

Threat	Mitigation
Unauthorized reads	RLS, membership checks, signed asset URLs
Spam and storage abuse	Rate limits, quotas, content validation
Malicious themes	JSON schema, allowlisted tokens, no executable imports
Realtime spoofing	Authenticated channels, bounded movement, server validation
XSS in chat/guestbooks	Escaping, safe rich-text renderer, sanitized

	links
Account compromise	Auth hardening, session management, audit trails
Asset copyright/privacy	Upload rules, reporting, takedown process

13. Free-tier deployment and operational limits

The operating target is ₱0/month for five testers. This is conditional: free plans have quotas, may change and may require verification or billing details. Verify each provider's current terms at deployment; never assume published allowances are permanent.

Proposed providers: Supabase free project for data/auth/storage/realtime; Cloudflare Pages for static frontend; GitHub free repositories/actions within allowances; Tauri and Phaser open source. Avoid requiring a custom domain or Microsoft Store registration for alpha.

Important cost controls: bundle common pixel assets, compress user media, cap room object count and visitors, throttle movement messages, unsubscribe on navigation, avoid large attachments, and record storage/realtime usage.

Backups: use periodic exports and tested restore procedures within provider permissions. Free services may not include automated backup guarantees; keep migrations and seed data in version control.

Operational checklist

- Record free-plan quotas and alerts before inviting testers.
- Create staging and production configurations without sharing production secrets.
- Set conservative upload and realtime budgets.
- Prepare rollback for frontend and schema changes.
- Test service suspension/recovery and local export.
- Do not commit to public launch without capacity and moderation review.

14. API, realtime events and data contracts

Client data access is mediated through typed repositories and authenticated Supabase queries/RPC functions. Keep authorization checks in database policies and trusted server-side functions where necessary. Schemas should be versioned.

Illustrative operations: createSpace, requestFriendship, sendMessage, listMessages(cursor), getHome, updateScene(expectedRevision), joinRoom, updateProfileAppearance, setHomePrivacy, enableHall.

Concurrency: editor saves include expected scene revision; reject conflicting writes and ask the user to reload or merge. Do not silently overwrite another authorized decorator's changes.

Events are ephemeral hints. After room.layout.updated, clients fetch the authoritative new revision; after reconnect, clients reload room membership and missed messages.

15. Accessibility, performance and quality

Target WCAG 2.2 AA for the conventional web UI where applicable. Provide keyboard access, readable contrast, visible focus, captions or text alternatives for meaningful pixel interactions, reduced-motion controls and an alternative non-spatial route to core features.

Performance targets are provisional: maintain responsive chat on ordinary laptops; use sprite atlases and viewport culling; limit concurrent particle effects; lazy-load Phaser and scene assets only when entering a room.

Testing layers: unit tests for validators/theme tokens; integration tests for repositories and RLS; end-to-end tests for account, messaging, home editing and permissions; five-person multiplayer sessions for reconnect, stale presence and conflict handling.

Alpha release gates

- No cross-user data exposure in policy tests.
- All five testers can send/receive persisted messages and recover after reconnect.
- Owners can save and reload homes without corrupting layouts.
- Unauthorized users cannot enter restricted homes or edit halls.
- World is optional; normal chat works if Phaser fails to load.
- Build and install instructions work on a clean Windows machine.

16. Delivery roadmap and acceptance criteria

Estimated solo-developer schedule: 8–12 weeks depending on availability, art creation and debugging. This is an estimate, not a deadline. Prioritize vertical slices that can be tested with the five friends.

Phase	Indicative window	Deliverable
0. Foundations	Week 1	Monorepo, design tokens, CI, Supabase schema/auth
1. Communication	Weeks 2–3	Friends, spaces, text chat, realtime reconnect
2. Pixel prototype	Weeks 4–5	One room, sprites, movement, join/leave, privacy
3. Home Studio	Weeks 6–7	Furniture catalog, editor, save/load, guestbook
4. Server Hall	Week 8	Optional hall, roles, portal and board
5. Polish and QA	Weeks 9–10	Desktop packaging, RLS tests, accessibility, limits
6. Alpha iteration	Weeks 11–12	Five-user feedback, fixes, usage measurement

Alpha definition of done

- A new invited member can join and customize their profile/home.
- Two or more friends can visit the same home and see movement.
- Server owner can create, enable and decorate an optional hall.

- Text chat works in and outside the world through shared backend data.
- All data operations are permission-checked and basic quotas enforced.
- The Windows app and hosted web app share consistent core functionality.

17. Public launch and scaling strategy

Stage A (five friends): closed alpha, single free backend project, bundled art, conservative quotas. Stage B (20–100 invited): load testing, metrics, moderation tools, backup/restore, asset processing and provider quota review. Stage C (public): public registration, discoverability, reporting operations, stronger abuse controls and measured infrastructure expansion.

When realtime traffic or room sizes outgrow free-tier design, introduce an authoritative multiplayer service without replacing the entire UI. Message storage may remain PostgreSQL while spatial state moves to a dedicated transport.

Consider a paid cloud budget only after measuring active rooms, messages/day, storage growth, egress and support workload. Revenue models are intentionally deferred; the foundational experience should not depend on premium cosmetics.

18. Risks, decisions and open questions

Risk / question	Current direction
Scope creep	Strict P0/P1/P2 backlog; defer voice and minigames
Realtime quotas	Throttle movement; validate with actual usage
Art production	Use original small starter set and clear sprite guidelines
Moderation load	Remain invite-only until policies/tools exist
Theme security	JSON allowlist; no arbitrary executable scripts
Editing conflicts	Optimistic revision locking
Public home visibility	Friends-only default; explicit user control
Voice feature	Post-alpha evaluation
Neighborhood feature	Future experiment, not MVP
Public business model	Undecided; do not build monetization yet

Decisions to validate with five testers

- Top-down versus isometric viewpoint (top-down is simpler initially).
- Whether homes support 16×12 tiles or another bounded map size.
- Preferred avatar scale, furniture footprint and art palette.
- Guestbook versus memory wall as first social object.
- How often users actually enter homes relative to chat.
- Whether halls should allow public visitors or members only.

19. Appendices: schemas, sample payloads and glossary

Example scene payload

```
{
 "schemaVersion": 1, "sceneId": "uuid", "kind": "home",
 "grid": {"width": 16, "height": 12, "tileSize": 32},
 "spawn": {"x": 2, "y": 2},
 "objects": [
  {"id": "uuid", "assetId": "desk_basic", "x": 5, "y": 4,
   "rotation": 0, "config": {"action": "open_profile_links"}}
 ],
 "revision": 3
}
```

Example theme payload

```
{
 "schemaVersion": 1, "name": "Realm Midnight",
 "appearance": "dark",
 "colors": {"background": "#101115", "accent": "#A78BFA"},
 "layout": {"density": "comfortable", "sidebarWidth": 260},
 "effects": {"reducedMotion": false, "avatarFrame": "glow"}
}
```

Glossary

Term	Definition
Home	Personal visitable pixel-art scene owned by a user
Hall	Optional shared pixel-art scene attached to a server
Scene	Versioned tile map plus placed objects and settings
Realm Object	Decorative or interactive furniture element
Identity Studio	Public profile customization editor
Theme Studio	Personal application appearance editor
Presence	Ephemeral status and room occupancy
RLS	PostgreSQL row-level security
Alpha	Invite-only initial release for five users

Implementation references

- Tauri documentation: https://tauri.app/

- Phaser documentation: https://docs.phaser.io/
- Supabase documentation: https://supabase.com/docs
- Cloudflare Pages: https://developers.cloudflare.com/pages/
- Tiled map editor: https://www.mapeditor.org/
- WCAG 2.2: https://www.w3.org/TR/WCAG22/

End of STXIC Realm Project Documentation v1.0.